

Constructor and Destructor



- 1. CONSTRUCTOR**
- 2. DEFAULT CONSTRUCTOR**
- 3. PARAMETERIZED CONSTRUCTOR**
- 4. COPY CONSTRUCTOR**
- 5. OVERLOADING CONSTRUCTOR**
- 6. DESTRUCTOR**

Constructor



- Special member function to initialize the objects of its class.
- Its name is same as the class name.
- It is invoked whenever the object is created.
- It construct the values of data members of the class so it is named as the constructor.

- Example

Class integer

```
{  
    int m,n;  
    public:  
    integer(); // constructor  
    declared  
  
    .....  
};
```

Constructor definition

```
integer:: integer()  
{  
    m=0;  
    n=0;  
}
```



- When object is created from class from class integer, it will be initialized automatically.
- Eg. `integer int1;`
- Here not only `int1` is created but also its data members `m` and `n` are initialized to zero.

Characteristics of Constructor



- They should be declared in the public section.
- They are called automatically when the object are created.
- They do not have return type even void.
- They have same name as the class name.
- They can have default arguments as the other function.

Default Constructor



- **They takes no parameters.**
- **They are called internally by the compiler whenever the object are created.**
- **There is no need to call it explicitly.**

Sample Program

```
#include<iostream.h>
#include<conio.h>
class integer
{
    int m,n;
public:
    integer()
    {
        m=0;
        n=0;
    }
}
```

```
void display()
{
    cout<<"m= "<<m<<" and n=
"<<n;
}
};
void main()
{
    clrscr();
    integer int1;
    int1.display();
    getch();
}
```

OUTPUT



```
m= 0 and n= 0_
```

Parameterized Constructor



- These are the constructor that take arguments.
- They initialized the object data members by the value which is passed as arguments.
- They are invoked when we pass the arguments to the object when they are being defined.
- Example: `integer int1(2,5);`

Sample Program

```
#include<iostream.h>
#include<conio.h>
class integer
{
    int m,n;
public:
    integer(int x, int y)
    {
        m=x;
        n=y;
    }
}
```

```
void display()
{
    cout<<"m= "<<m<<"
    and n= "<<n;
}

};

void main()
{
    clrscr();
    integer int1(5,6);
    int1.display();
    getch();
}
```

OUTPUT



```
m= 5 and n= 6_
```

Copy Constructor



- It is used to declare and initialized an object from another object.
- It takes reference to an object of the same class as itself as an arguments.
- Example:

```
integer I1;  
integer I2(I1);
```

Sample Program

```
#include<iostream.h>
#include<conio.h>
class integer
{
    int m,n;
public:
    integer(integer &x)
    {
        m=x.m; n=x.n;
    }
}
```

```
integer()
{
    m=100; n=100;
}

void display()
{
    cout<<"m= "<<m<<"
    and n= "<<n<<endl;
}
};
```



```
void main()
{
    clrscr();
    integer int1;
    int1.display();
    integer int2(int1);
    int2.display();
    getch();
}
```

OUTPUT



```
m= 100 and n= 100  
m= 100 and n= 100
```

Overloading Constructor



- Constructor overloading is the process of defining more than one constructor in the same class.
- C++ permits us to use multiple constructor in the same class.

Sample Program

```
#include<iostream.h>
#include<conio.h>
class integer
{
    int m,n;
public:
    integer(integer &x)
    {
        m=x.m;
        n=x.n;
    }
}
```

```
integer()
{
    m=0;
    n=0;
}
integer(int x, int y)
{
    m=x;
    n=y;
}
```



```
void display()
{
    cout<<"m= "<<m<<"
    and n= "<<n<<endl;
}
};
```

```
void main()
{
    clrscr();
    integer int1;
    integer int2(400,500);
    integer int3(int2);
    int1.display();
    int2.display();
    int3.display();
    getch();
}
```

OUTPUT



```
m= 0 and n= 0  
m= 400 and n= 500  
m= 400 and n= 500
```

Destructor



- It is used to destroy the objects created by the constructor.
- It is called for the class object whenever it passes the scope in the program.
- Whenever new is used in the constructor to allocate the memory delete should be used in the destructor to free the memory for future use.

Characteristics of Destructor



- It has same name as the class name but is preceded by tilde (~) sign.
- It has no return type and do not take any arguments.
- It can not be overloaded.
- It is called whenever the object get out of its scope.

Sample Program

```
#include<iostream.h>
```

```
class integer
```

```
{
```

```
    int m,n;
```

```
    public:
```

```
    integer()
```

```
{
```

```
        m=0;
```

```
        n=0;
```

```
        cout<<"Default  
        Constructor is  
        called"<<endl;
```

```
integer(int x, int y)
```

```
{
```

```
    m=x;
```

```
    n=y;
```

```
    cout<<"Parameterize  
    d Constructor is  
    called"<<endl;
```

```
}
```

```
~integer()
```

```
{
```

```
    cout<<"Object is  
    destroyed"<<endl;
```

```
}
```



```
void display()
{
    cout<<"m=
    "<<m<<" and n=
    "<<n<<endl;
}
};
```

```
void main()
{
    clrscr();
    {
        integer int1;
        int1.display();
    }
    {
        integer int2(2,5);
        int2.display();
    }
}
```

OUTPUT



```
Default Constructor is called  
m= 0 and n= 0  
Object is destroyed  
Parameterized Constructor is called  
m= 2 and n= 5  
Object is destroyed
```